Intern : comparaison VM et conteneurs

Configuration et protocole

Référence VM du 12 octobre et campagne Docker du 14 octobre 2026, sur le même poste Linux, noyau 6.19.14-108.fc42.x86_64, 12 processeurs logiques. VirtualBox 7.2.8_RPMFUSIONr173730 ; Docker Engine 28.3.3 rootless, Compose 2.39.4, réseau VPNKit et stockage fuse-overlayfs. Le moteur Docker fonctionne sous Linux sans VM intermédiaire.

Trois arrêts/démarrages complets, trente secondes au repos, dix GET par série sur `/api/v1/offers/` via le proxy 8080 : quatre offres fictives, urllib, connexion neuve, réponse et JSON entièrement lus. Cache disque hôte conservé. Médiane des trois séries ; pour la latence, médiane des trois médianes de dix GET. Le seuil de démarrage est le premier catalogue HTTP 200 non vide.

Tableau comparatif chiffré

Grandeur	Trois VM	Trois conteneurs	Écart Docker / VM
Démarrage jusqu'au catalogue	20.473 s	49.844 s	+143.5 %
Mémoire RSS côté hôte	1.454 Gio	0.568 Gio	-60.9 %
Disque : taille des fichiers	10.783 Gio	0.809 Gio	-92.5 %
Disque : blocs alloués	10.755 Gio	0.864 Gio	-92.0 %
Latence de consultation	62.431 ms	83.385 ms	+33.6 %
Mise en place humaine depuis zéro	Non chronométrée	Non chronométrée	Non calculable

RSS : trois processus VM, hors VBoxSVC ; côté Docker, processus des conteneurs, daemon, containerd, shims, FUSE, réseau et publication rootless inclus. Disque : les snapshots VM sont inclus ; les couches Docker partagées sont comptées une fois, avec les couches modifiables et le volume PostgreSQL. Caches de construction et archives d'export exclus. Les octets exacts restent dans les fichiers bruts. Le temps humain initial ne peut pas être reconstitué.

Matrice de joignabilité réellement testée

Depuis	Proxy :80	Métier :8000	Données :5432
Hôte	Oui, via 8080	Échec attendu	Échec attendu
Proxy	Sans objet	Oui	Échec attendu
Métier	Oui	Sans objet	Oui
Données	Échec attendu	Oui	Sans objet

Neuf liaisons conformes, dont quatre refus. Tous les services étaient sains. Les destinations interdites ont été testées par leur IP réelle, pas seulement par un nom absent du DNS : délais dépassés ou réseau inaccessible. Le proxy et la base n'ont aucun réseau partagé et seul le proxy publie un port.

Séries Docker retenues

Série	Démarrage	RSS moteur inclus	Médiane de dix GET
1	49.844 s	0.568 Gio	83.385 ms
2	42.337 s	0.592 Gio	88.268 ms
3	51.412 s	0.567 Gio	75.539 ms

1. De combien les résultats diffèrent-ils ?

Le démarrage Docker prend 29.371 s de plus (+143.5 %) et la consultation 20.955 ms de plus (+33.6 %). La mémoire RSS diminue de 0.886 Gio (-60.9 %). Les blocs disque alloués diminuent de 9.892 Gio (-92.0 %). Cette configuration économise donc mémoire et stockage, mais démarre et répond plus lentement. Aucun gagnant universel n'est déduit de ces observations.

2. Quelles propriétés techniques expliquent ces écarts ?

Le noyau partagé évite trois systèmes invités et leurs services : cela explique une partie de l'économie de mémoire et de disque. Les images ne conservent que les fichiers nécessaires ; les couches communes sont partagées. L'image métier reste sous 389 Mo, sans Git, uv ni cache uv ; le proxy reste sous 56 Mo. Le disque VM contient aussi des snapshots : tout l'écart ne vient pas du noyau.

La lenteur observée doit être interprétée avec la configuration réelle : stockage FUSE, réseau rootless et démarrage conditionné par les sondes de santé. Ces mécanismes ajoutent des opérations et des attentes aux trois services. Ce sont des explications plausibles, pas des contributions individuellement mesurées. Les VM ont 1/2/2 vCPU et 1/2/2 Gio ; les conteneurs ne sont pas bornés aux mêmes quotas, Nginx utilise ses workers automatiques. Python passe de 3.12.3 à 3.12.15 et Nginx de 1.24.0 à 1.28.0 ; Django, Gunicorn et PostgreSQL restent dans les versions de référence. Un service de logos Python diffère également de l'empreinte VM. Le scénario fictif est vérifié, mais ces différences interdisent d'attribuer causalement tout l'écart à la seule technique d'isolation.

3. Que perd-on par rapport aux machines virtuelles ?

Les conteneurs partagent le noyau hôte : l'isolation repose sur ses mécanismes logiciels, tandis que les VM possèdent un noyau invité séparé derrière l'hyperviseur. Une défaillance du noyau partagé concerne toute la pile. Un conteneur Linux n'apporte pas un noyau Windows ni un autre noyau Linux. Le mode rootless réduit les privilèges du daemon sans créer la frontière d'une VM. Les images ne remplacent pas les sauvegardes des volumes ni les snapshots : la reconstruction restaure le logiciel, pas les données modifiées.

Contrôle des erreurs fréquentes et organisation

Le volume PostgreSQL persiste ; ses scripts ne sont joués que sur un volume vide. Les migrations manquantes font échouer explicitement le démarrage. Les dépendances attendent la santé et `POSTGRES_HOST=donnees` évite localhost. Les bases sont versionnées et légères, les dépendances précèdent le code, les caches restent hors de l'image finale et aucun port SQL n'est publié. La reconstruction a réellement réutilisé le cache de couches.

Les sources sont regroupées dans `docker/` : `metier.Dockerfile`, `proxy.Dockerfile`, `compose.yaml`, `.dockerignore`, `PROCEDURE.md`. Les procédures et preuves VM restent dans `deploiement/`. Sources chiffrées : VM, Docker, matrice, contrôles SQL, recette navigateur. Référence technique : Docker rootless.